

Полное описание проекта

TAKT Industrial Risk Layer (MVP)

Версия пакета: 0.6.23

Документ сформирован (UTC): 2026-05-02 09:01 UTC

Репозиторий: D:\TAKT\takt-industrial-risk-layer

Документ объединяет сведения из README, docs/product_boundary.md, конфигурации config/risk_weights.yaml и структуры исходного кода. Назначение — дать целостную картину продукта для архитекторов, инженеров эксплуатации и аналитиков ИБ.

Структура документа

1. Контекст и нормативная база (ТЗ MVP 2.0)
2. Цели, зона ответственности и границы продукта
3. Архитектура: слои L1–L4 и правила зависимостей
4. Доменный слой L1: сущности, движки, инварианты
5. Прикладной слой L2: сценарии использования
6. REST API: обзор маршрутов и семантика
7. Модель кейса, очередь событий и слияние (alert fatigue)
8. Конфигурация YAML и переменные окружения
9. Хранилища, персистентность SQLite
10. Экспорт, интеграции SIEM, backtest
11. Наблюдаемость, безопасность HTTP, лимиты
12. Сборка, Docker, Kubernetes
13. Качество: тесты, CI, SBOM, extras пакета
14. Направления дальнейшей разработки

1. Контекст и нормативная база

Проект реализует автономный аналитический компонент экосистемы ТАКТ — «Industrial Risk Layer» для критической информационной инфраструктуры и АСУ ТП. Концепция и требования выведены из T3 MVP 2.0 (слои L1–L5, 26 инвариантов, Chaos Predictor с целевой константой Фейгенбаума $\delta \approx 4.669$, Causal Mesh, Risk/XAI). Сопутствующие материалы спринтов и чек-листов размещаются в каталоге D:\ТАКТ\docs (извлечённые тексты из исходных DOCX). Кодовая база следует этим документам на уровне MVP.

2. Цели, зона ответственности и границы продукта

В зоне ответственности

Приём и нормализация событий из JSON/CSV-конвейеров. Оценка качества данных (DQ), учёт контекста (заявки, окна работ, фазы), ритма опроса и топологии (Causal Mesh). Агрегация факторов в индекс риска и формирование объяснимых выводов (XAI) без «чёрного ящика». Управление кейсами с участием человека (HITL), экспорт отчётов и карточек, воспроизводимый backtest.

Строгие запреты

ТАКТ не является SIEM, XDR, СКЗИ, системой активного управления или автоматической блокировки. В слое L1 Domain запрещены криптография, вызовы ОС/сети, ORM, FastAPI и прочая инфраструктура. MVP не выполняет активное управление оборудованием АСУ ТП.

Задел

Порты на границах L2/L4 допускают подключение постквантовой криптографии и внешних адаптеров без изменения доменных сущностей и правил.

3. Архитектура: слои L1–L4 и правила зависимостей

L1 Domain — каталог src/takt/domain: сущности (событие, кейс, актив, заявки), движки риска, данных, топологии, XAI, Chaos Predictor, контекст и фазы; каталог 26 инвариантов и модуль invariants/evaluator с расширенными правилами; порты репозитория кейсов и baseline. Слой не импортирует FastAPI, uvicorn и инфраструктуру.

L2 Application — src/takt/application/use_cases: оркестрация домена без HTTP. Контракт import-linter: прикладной слой не зависит от infrastructure и interface_adapters.

L3 Interface adapters — src/takt/interface_adapters/api: приложение FastAPI, маршруты, валидация входа, отображение доменных объектов в DTO ответов.

L4 Infrastructure — src/takt/infrastructure: загрузка YAML, фабрики хранилищ, SQLite, импорт CSV, экспорт PDF (fpdf2), webhook SIEM (httpx), HTTP middleware (идентификатор запроса, gzip, CORS, HSTS, rate limit, лимит тела, метрики Prometheus).

4. Доменный слой L1: сущности, движки, инварианты

Ключевые модули

engines: risk_engine — агрегация факторов; data_quality — DQ и частичная наблюдаемость; causal_mesh — граф и паттерны (в т.ч. обход jump-сервера); chaos_predictor — связь с ритмом опроса; context_matcher, phase_time_tagger — контекст ТО и временные фазы; xai — человекочитаемые объяснения; alert_fatigue — ключ слияния burst_fingerprint (source|asset_id|operation), отдельные кейсы при разных source.

Каталог из 26 инвариантов (id, block_key, заголовок)

- polling_jitter [rhythm] — Аномальный джиттер интервалов опроса
- illegal_function_code [rhythm] — Недопустимый код функции / IEC ASDU type id

- `payload_length_drift` [rhythm] — Дрейф размера полезной нагрузки
- `request_reply_dissonance` [rhythm] — Расхождение запрос/ответ по протоколу
- `jump_server_bypass` [topology] — Доступ к ПЛК/критузлу минуя jump-сервер
- `new_node_airgap` [topology] — Новый узел в сегменте air-gap
- `lateral_movement` [topology] — Признаки lateral movement
- `reconnaissance` [topology] — Разведка: скан, SNMP walk, топология
- `untrusted_ip_admin` [identity] — Админ-действия с недоверенного IP
- `brute_force` [identity] — Серия неуспешных аутентификаций
- `out_of_shift_access` [identity] — Администрирование вне штатной фазы/смены
- `protocol_escalation` [identity] — Эскалация «тяжёлого» протокола по активу
- `conflict_logic` [physics] — Конфликт логики управления / взаимоисключающие команды
- `physical_invariant_breach` [physics] — Нарушение физического инварианта процесса
- `blind_command` [physics] — Команда записи без предшествующего чтения/опроса
- `cyclic_service_crash` [physics] — Циклический отказ сервиса исполнения
- `log_wiping` [integrity] — Очистка журналов / аудита
- `runtime_config_change` [integrity] — Изменение конфигурации на исполнении
- `c2_external_dns` [integrity] — Подозрительный внешний DNS / C2-паттерн
- `trust_index_drop` [integrity] — Просадка индекса доверия источника
- `stale_data` [data_hitl] — Устаревшие данные в конвейере
- `telemetry_gap` [data_hitl] — Разрыв телеметрии / пропуск выборок
- `source_reputation_drift` [data_hitl] — Дрейф репутации источника
- `expert_dissonance` [data_hitl] — Расхождение с экспертной оценкой / HITL
- `alert_fatigue_guard` [data_hitl] — Слияние серии срабатываний (alert fatigue)
- `context_dissonance` [data_hitl] — Расхождение с контекстом ТО / заявок
- `polling_chaos_feigenbaum` [rhythm] — Хаос опроса — стремление ритма к δ Фейгенбаума

Ключи блоков для фильтрации API: rhythm, topology, identity, physics, integrity, data_hitl. GET /invariants отдаёт полный справочник с метаданными для UI и интеграций.

5. Прикладной слой L2: сценарии использования

`AssessRiskUseCase` — оценка риска с учётом `InvariantContext` из конфигурации и последних событий.

`ProcessEventUseCase` — ingest одного события или элемент пакетной обработки: обогащение, DQ, инварианты, обновление/создание кейса.

`SubmitCaseDecisionUseCase` — смена статуса кейса; при `EXPECTED_BEHAVIOR` обновляется baseline пар актив+операция.

`RunBacktestUseCase` — прогон фикстуры (например `tests/fixtures/plc_polling_demo.csv`) с теми же рёбрами графа и `ingest_trust`, что у работающего API.

6. REST API: обзор маршрутов и семантика

Система и документация

GET/HEAD /live — liveness без обращения к БД. GET/HEAD /ready — readiness: чтение хранилища кейсов и baseline (503 при сбое). GET/HEAD /health — расширенная диагностика (версия пакета, Python, лимиты, SQLite, метрики, CORS, HSTS, gzip, rate limit, openapi servers, счётчики кейсов и т.д.). GET/HEAD /metrics — текст Prometheus при TAKT_METRICS и установленном prometheus-client. GET/HEAD /openapi.json, UI /docs и /redoc. Заголовки X-Request-ID, X-Process-Time; опционально API-ключ X-TAKT-API-Key.

Каталог и демо-данные

GET /invariants — все InvariantId с фильтром ?block_key=. GET /catalog/event-sources — допустимые source и коэффициенты ingest_trust. GET /topology/demo-graph — jump_host, plc_hosts, demo_graph_edges, признак has_jump_bypass_pattern.

Ingest и оценка

POST /assess/demo — демо-оценка с сохранением кейса и слиянием по burst_fingerprint. POST /events — одно событие (окно последних событий в памяти до 64, в контекст оценки — последние 5). POST /events/batch — до 100 событий за запрос, ответ с итогами по позициям. Нормализация IEC-104 полей задаётся iec104_type_aliases и iec104_disallowed_type_ids в YAML.

Кейсы

GET /cases — список с богатым набором фильтров (источник, время создания, класс риска, score, число событий, текстовые подстроки, инварианты, DQ и др.), сортировка, limit до 1000, X-Total-Count, Link rel=next/prev. GET /cases/{id} — полная карточка. GET /cases/stats — агрегаты по статусам, классам, источникам, DQ. GET /cases/export/full.json и POST /cases/import/full.json — выгрузка/импорт до 10000 карточек (транзакция в SQLite).

Решение по кейсу и качество данных

POST /cases/{id}/decision — JSON со статусом (TRIAGE и др.). GET /data-quality — снимок после последней оценки/ingest.

Экспорт и интеграции

GET /cases/{id}/export.pdf — «паспорт инцидента»; при export.pdf_unicode_font в YAML указан путь к .ttf, в PDF корректна кириллица в XAI. GET /cases/{id}/export/siem.json — полезная нагрузка для SIEM (last_event_source, invariant_hits, invariant_details, data_quality). POST /integrations/siem/forward и .../forward/async — отправка в webhook по allowlist из siem_webhook; при пустом allowlist разрешены только loopback URL.

Аналитика

POST /backtest/fixture — прогон демо CSV с конфигурацией API. Маршруты в Swagger сгруппированы по тегам: System, Catalog, Ingest, Cases, Export, Integrations, Analytics.

7. Модель кейса, очередь событий и слияние

Кейс накапливает историю событий, срабатывания инвариантов, risk_score и risk_class (LOW/MEDIUM/HIGH/CRITICAL), XAI-summary и показатели DQ. Слияние при совпадающем burst_fingerprint объединяет invariant_hits, берёт максимум risk_score, при равенстве — более высокий risk_class, обновляет пояснения с последнего события. Разные source при той же паре актив+операция дают разные кейсы — отражение доверия к каналу данных.

8. Конфигурация YAML и переменные окружения

Файл config/risk_weights.yaml

Весы факторов Risk = F(R, G, C, U, DQ): rhythm, graph, context, user, data_quality; пороги mandelbrot_entropy_cap, dq_degraded_threshold, feigenbaum_target, eps_soft_cap. Секции topology, invariants, enrichment, ingest_trust.by_source, storage (backend memory|sqlite, sqlite_path), siem_webhook (allowed_url_prefixes, retries, backoff_sec), export.pdf_unicode_font.

Переменные окружения (см. README и .env.example)

TAKT_CONFIG, TAKT_STORAGE, TAKT_SQLITE_PATH, TAKT_SQLITE_BUSY_TIMEOUT_MS, TAKT_API_KEY, TAKT_CORS_ORIGINS, TAKT_HSTS_MAX_AGE, TAKT_HSTS_PRELOAD, TAKT_LOG_LEVEL, TAKT_REQUEST_ID_HEADER, TAKT_SLOW_REQUEST_LOG_SEC, TAKT_MAX_REQUEST_BODY_MB, TAKT_RATE_LIMIT_PER_MIN, TAKT_RATE_LIMIT_MAX_IPS,

TAKT_RATE_LIMIT_IP_HEADER, TAKT_CATALOG_CACHE_MAX_AGE_SEC, TAKT_METRICS, TAKT_OPENAPI_SERVER_URL, TAKT_BUILD_REVISION. Uvicorn ≥ 0.30 поддерживает `--env-file .env` для локального запуска.

9. Хранилища и персистентность SQLite

Режим `memory` по умолчанию: кейсы и `expected_behavior` в процессе; окно последних событий всегда в RAM. Режим `sqlite`: файловая БД с WAL, PRAGMA `busy_timeout` (по умолчанию 5000 мс, переопределение через `env`), таблицы кейсов, `baseline` и `app_metadata` (`schema_version`). При остановке приложения — корректное закрытие и `wal_checkpoint`. Импорт полного JSON выполняется в одной транзакции.

10. Экспорт, интеграции SIEM, backtest

PDF строится `fpdf2`; без Unicode-шрифта поля с кириллицей деградируют в latin-1 substitution. Webhook использует политику URL из конфигурации и ретрай с `backoff`. Backtest воспроизводит демо-поток для валидации логики без продакшен-трафика.

11. Наблюдаемость, безопасность HTTP, лимиты

Метрики: `takt_http_requests_total`, `takt_http_request_duration_seconds`, `takt_build_info`, при лимите — отказов и tracked IPs. Gzip ответов от порога 512 байт. Security headers: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy; опционально HSTS. Rate limit по IP (или доверенному заголовку) в минуту; публичные пути `/live`, `/ready`, `/health`, `/metrics`, документация — без квоты. Ответы 403/411/413/429 с телом `MiddlewareErrorJson` и `request_id`. Кэш Cache-Control для каталожных GET; `/cases` — `private, no-store`.

12. Сборка, Docker, Kubernetes

Локально: `pip install -e ".[dev]"`, `pytest`, `uvicorn takt.interface_adapters.api.main:app`. Dockerfile: Python 3.13 slim; optional build-arg TAKT_BUILD_REVISION; HEALTHCHECK на HEAD `/ready`; graceful shutdown `uvicorn` с таймаутом 15 с (для K8s рекомендуется `terminationGracePeriodSeconds  $\geq 20$`). Примеры kube-проб: `liveness GET /live`, `readiness GET /ready` на порт 8090. `docker-compose` в репозитории с согласованным healthcheck.

13. Качество: тесты, CI, SBOM, extras пакета

`pytest` покрывает домен, use cases, API, middleware, SQLite, экспорт и интеграции. `import-linter` фиксирует границы слоёв. GitHub Actions: сборка образа (без push, кэш BuildKit) и `pytest` на Python 3.11–3.14; Dependabot для `pip` и actions. Скрипт `scripts/generate_sbom.py` формирует `dist/sbom-components.json` (упрощённый CycloneDX-lite). Опциональные extras: `dev` (`pytest`, `import-linter`, `fpdf2`, `prometheus-client`), `metrics`, `export`.

14. Направления дальнейшей разработки

По README: расширение покрытия инвариантами из полного ТЗ, углублённый разбор IEC-104 (сейчас нормализация и подсказки), асинхронный backtest на 100k+ событий, целевые стенды (Astra/Baikal).